day-01-interview-qa.md

Tags

Question 1: list vs tuple vs set vs dict

Your answer is mostly right. Small fixes:

- Set: say “no index, no order” (you did say no index).
- Dict: keys must be unique, and you look things up by key.

Simple version with your job data:

Type	Use it when	Example
list `[]`	order matters and it can change	`jobs` (all your job listings)
tuple `()`	fixed values that shouldn’t change	`("AI Engineer", 95000)`
set `{}`	unique values, fast “is it there?” check	`my_skills`
dict `{k: v}`	look up a value by name	`job["salary"]`

Question 2: why is `x in my_set` faster?

Your answer needs a correction. Having no duplicates is not the reason.

Correct answer: a list checks items **one by one** from the start. A set jumps **straight** to the item, like using a book’s index instead of reading every page.

Try it yourself:

python

```
import timeitmy_list = list(range(1_000_000))my_set = set(my_list)print(timeit.timeit(lambda: 999_999 in my_list, number=100))print(timeit.timeit(lambda: 999_999 in my_set, number=100))
```

The set version should be thousands of times faster.

Question 3: `sort()` vs `sorted()`

Your answer is correct. Add two points:

- `sort()` works only on lists. `sorted()` works on any iterable and always gives back a list.
- `sort()` returns `None`. A common bug is `jobs = jobs.sort(...)`, which makes `jobs` equal to `None`.

Question 4: `dict["key"]` vs `dict.get("key")`

Your answer is correct. Small fixes: the error is called `KeyError`, and `None` has a capital N. You missed the **default value**:

python

```
job.get("benefits", "not listed")   # "not listed" instead of None
```

Question 5: list comprehension vs generator expression

Correct. Add one missing point: a generator can only be used **once**.

python

```
gen = (x for x in range(3))print(list(gen))   # [0, 1, 2]print(list(gen))   # []  already used up
```

And the numbers: for a million items, the list is about 8 MB, while the generator is a couple of hundred bytes.

Question 6: iterable, iterator, generator

Good, and the bookmark idea is correct. Add the missing words `next()` and `yield`:

- iterable: can be looped over (`[1, 2, 3]`)
- iterator: the bookmark, you move it with `next()` (`iter([1, 2, 3])`)
- generator: an iterator you build with a function using `yield` (`countdown(3)`)

Question 7: variable shadowing (you left it blank)

Simple answer: shadowing is when your variable has the **same name as something that already exists**, so yours hides the original.

python

```
sum = 0for s in [10, 20]:    sum = sum + sprint(sum)
# 30print(sum([1, 2, 3]))  # TypeError: 'int' object is not c
allable
```

After `sum = 0`, `sum` is a number, not the built-in function. In your `salary_stats` it was only hidden inside that function, but it's a bad habit. Use `total` instead.

Question 8: count and group (blank)

python

```
Counter(skill_list_all).most_common(3)   # count itemsgroups
= defaultdict(list)                # group itemsfor job in job
s:    groups[job["company"]].append(job["title"])
```

Say it simply: `Counter` is a tally sheet, and `defaultdict(list)` makes a new empty basket whenever it sees a new key.

Question 9: shallow vs deep copy

Your answer needs a correction. Shallow copy **does** copy a dict. The problem is only with things **inside** it.

Correct answer: a shallow copy copies only the outer layer. Lists and dicts inside are still shared. A deep copy copies everything, so nothing is shared.

python

```
copy_a = jobs.copy()             # shallow: the 5 job dicts ar
e sharedcopy_b = copy.deepcopy(jobs)    # deep: fully separat
e
```

In JS: `[...jobs]` is shallow and `structuredClone(jobs)` is deep.

Question 10: flatten a list of lists (blank)

python

```
flat = [skill for job in jobs for skill in job["skills"]]
```

Read it as: “for each job, for each skill in it, give me the skill”. This is the same loop order as the nested loop you wrote.

Question 11: when NOT to use a list comprehension

You’re half right. For a huge file (6 GB), a list comprehension loads everything into memory. Use a generator expression or a normal loop instead.

Add these reasons:

- It’s too complicated to read (more than two loops or conditions).
- You’re doing an **action** like `print`, not building a value. You hit this in Exercise 13.
- Building something like your grouping dict, where a loop is clearer.

Question 12: `a, b = b, a + b` (blank)

Simple answer: it computes both new values first using the **old** `a` and `b`, then assigns them together.

Two cups: `a` has water and `b` has juice.

- `a, b = b, a + b` is like using both cups to mix the next drink first, then swapping them.
- `a = b` followed by `b = a + b` pours juice into the water cup first, so the old water is gone when you need it. Then `a + b` is just `b + b`, which is why you got 0, 1, 2, 4, 8...